

Ex.No.	List of Experiments	Page no	Mark	Staff sign.
1	Working with Pandas data frames			
2	Basic plots using Matplotlib			
3A	Frequency distributions			
3B	Averages			
3C	Variability			
4A	Normal curves			
4B	Correlation and scatter plots			
4C	Correlation coefficient			
5	Simple Linear Regression			
6	Z-test			
7	T-test			
8	ANOVA			
9	Building and validating linear models			
10	Building and validating logistic models			

EXP NO: 1	Working with Pandas data frames
Date:	

AIM: To work with Pandas data frames.

ALGORITHM:

Step1: Start

Step2: import pandas module

Step3: Create a dataframe using the dictionary

Step4: Print the output

Step5: Stop

PROGRAM:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)

print(df.head())

filtered_df = df[df['Age'] > 30]

print(filtered_df)

df['Senior'] = df['Age'] > 30

print(df)

grouped_df = df.groupby('City')['Age'].mean()

print(grouped_df)
```

OUTPUT:

```
      Name Age      City
0  Alice  25  New York
1   Bob  30  Los Angeles
2 Charlie 35   Chicago
      Name Age      City
2 Charlie 35   Chicago
      Name Age      City Senior
0  Alice  25  New York  False
1   Bob  30  Los Angeles False
2 Charlie 35   Chicago   True
City
Chicago      35.0
Los Angeles  30.0
New York     25.0
Name: Age, dtype: float64
```

RESULT:

Thus the working with Pandas data frames was successfully completed.

EXP NO: 2	Basic plots using Matplotlib
Date:	

AIM:

To draw basic plots in Python program using Matplotlib.

ALGORITHM:

Step1: Start

Step2: import Matplotlib module

Step3: Create a Basic plots using Matplotlib

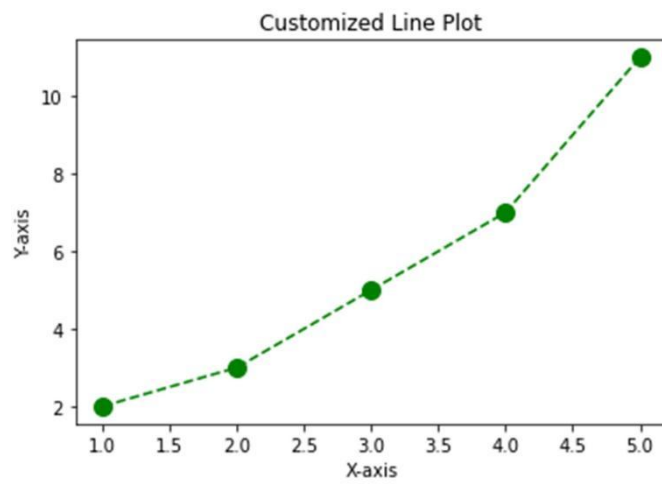
Step4: Print the output

Step5: Stop

PROGRAM:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
plt.plot(x, y, color='green', linestyle='--', marker='o', markersize=10)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Customized Line Plot')
plt.show()
```

OUTPUT:



RESULT:

Thus the basic plots using Matplotlib in Python program was successfully completed.

EXP NO: 3A	Frequency distributions
Date:	

AIM :

To write a python program to the frequency distribution in jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import the python library modules

Step 3: Write the code the frequency distribution

Step 5: Print the result

Step 6: Stop the program

PROGRAM:

```
import pandas as pd

data = [1, 2, 2, 3, 4, 4, 4, 5, 6, 7, 7, 7, 7]

series = pd.Series(data)

frequency = series.value_counts()

print(frequency)
```

OUTPUT:

```
7    4
4    3
2    2
1    1
3    1
5    1
6    1
dtype: int64
```

RESULT:

Thus the python program to the frequency distribution in jupyter notebook was written and executed successfully.

EXP NO: 3B	Averages
Date:	

AIM :

To write a python program to find an average in jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import the python library modules

Step 3: Write the code the average

Step 5: Print the result

Step 6: Stop the program

PROGRAM:

```
import numpy as np
import pandas as pd
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
mean_pandas = pd.Series(data).mean()
print(f'Mean (Pandas): {mean_pandas}')
mean_numpy = np.mean(data)
print(f'Mean (NumPy): {mean_numpy}')
median = pd.Series(data).median()
print(f'Median: {median}')
mode = pd.Series(data).mode()
print(f'Mode: {mode}')
```

OUTPUT:

Mean (Pandas): 5.5

Mean (NumPy): 5.5

Median: 5.5

Mode: 0 1

1 2

2 3

3 4

4 5

5 6

6 7

7 8

8 9

9 10

dtype: int64

RESULT:

Thus the python program to the average in jupyter notebook was written and executed successfully.

EXP NO: 3C	Variability
Date:	

AIM :

To write a python program to find an average in jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import the python library modules

Step 3: Write the code the average

Step 5: Print the result

Step 6: Stop the program

PROGRAM:

```
import numpy as np

import pandas as pd

data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

range_value = max(data) - min(data)

print(f"Range: {range_value}")

variance_pandas = pd.Series(data).var()

print(f"Variance (Pandas): {variance_pandas}")

std_dev_pandas = pd.Series(data).std()

print(f"Standard Deviation (Pandas): {std_dev_pandas}")

variance_numpy = np.var(data)

print(f"Variance (NumPy): {variance_numpy}")

std_dev_numpy = np.std(data)

print(f"Standard Deviation (NumPy): {std_dev_numpy}")
```

OUTPUT:

Range: 9

Variance (Pandas): 9.166666666666666

Standard Deviation (Pandas): 3.0276503540974917

Variance (NumPy): 8.25

Standard Deviation (NumPy): 2.8722813232690143

RESULT:

Thus the computation for variance was successfully completed.

EXP NO: 4A	Normal curves
Date:	

AIM :

To create a normal curve using python program

ALGORITHM:

Step 1: Start the program

Step 2: Import packages numpy and matplotlib

Step 3: Create the distribution

Step 4: Visualizing the distribution

Step 5: Stop the program

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

mu = 0

sigma = 1

data = np.random.normal(mu, sigma, 1000)

plt.hist(data, bins=30, density=True, alpha=0.6, color='g')

xmin, xmax = plt.xlim()

x = np.linspace(xmin, xmax, 100)

p = 1/(sigma * np.sqrt(2 * np.pi)) * np.exp(-0.5 * ((x - mu) / sigma)**2)

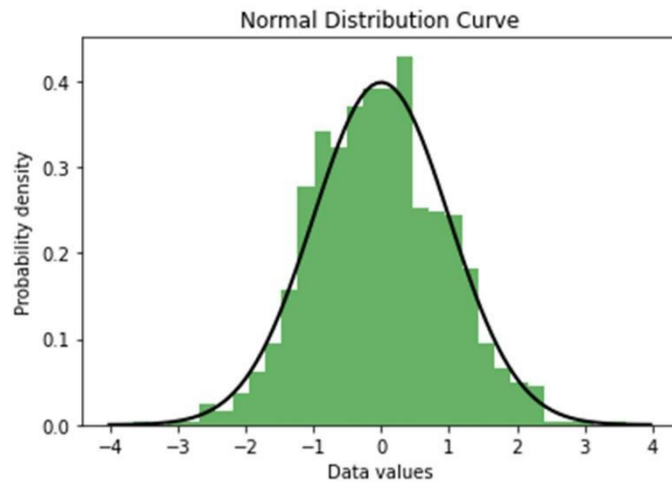
plt.plot(x, p, 'k', linewidth=2)

plt.xlabel('Data values')

plt.ylabel('Probability density')

plt.title('Normal Distribution Curve')
```

```
plt.show()
```

OUTPUT:**RESULT:**

Thus the normal curve using python program was successfully completed.

EXP NO: 4B	Correlation and scatter plots
Date:	

AIM :

To write a python program for correlation with scatter plot.

ALGORITHM:

Step 1: Start the Program

Step 2: Create variable y1, y2

Step 3: Create variable x, y3 using random function

Step 4: plot the scatter plot

Step 5: Print the result

Step 6: Stop the process

Program:

```
import matplotlib.pyplot as plt

import numpy as np

x = np.random.rand(100)

y = 2 * x + np.random.normal(0, 0.1, 100)

plt.scatter(x, y, alpha=0.7)

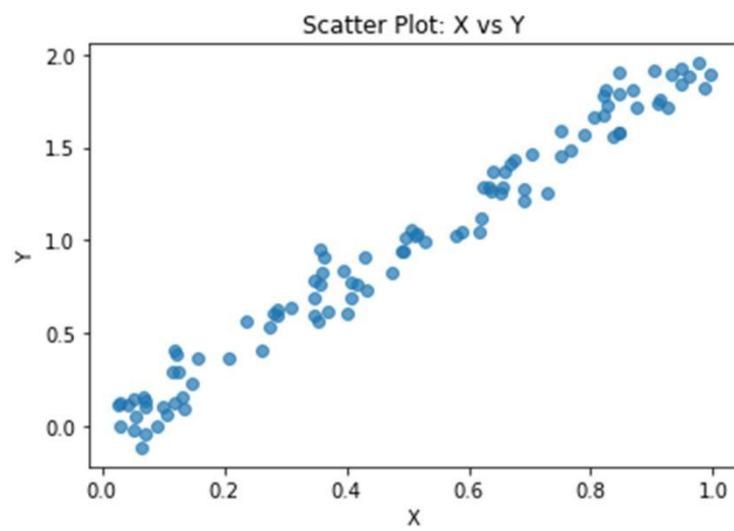
plt.xlabel('X')

plt.ylabel('Y')

plt.title('Scatter Plot: X vs Y')

plt.show()
```

OUTPUT:



Result:

Thus the Correlation and scatter plots using python program was successfully completed.

EXP NO: 4C	Correlation coefficient
Date:	

Aim:

To write a python program to compute correlation coefficient.

ALGORITHM

Step 1: Start the Program

Step 2: Import math package

Step 3: Define correlation coefficient function

Step 4: Calculate correlation using formula

Step 5: Print the result

Step 6 : Stop the process

PROGRAM:

```
import numpy as np

import pandas as pd

x = np.random.rand(100)

y = 2 * x + np.random.normal(0, 0.1, 100)

correlation_numpy = np.corrcoef(x, y)[0, 1]

print(f'Correlation Coefficient (NumPy): {correlation_numpy}')

df = pd.DataFrame({'x': x, 'y': y})

correlation_pandas = df.corr().loc['x', 'y']

print(f'Correlation Coefficient (Pandas): {correlation_pandas}')
```

OUTPUT:

Correlation Coefficient (NumPy): 0.979876345

Correlation Coefficient (Pandas): 0.979876345

Result:

Thus the computation for correlation coefficient was successfully completed.

EXP NO: 5	Simple Linear Regression
Date:	

AIM:

To write a python program for Simple Linear Regression

ALGORITHM:

Step 1: Start the Program

Step 2: Import numpy and matplotlib package

Step 3: Define coefficient function

Step 4: Calculate cross-deviation and deviation about x

Step 5: Calculate regression coefficients

Step 6: Plot the Linear regression and define main function

Step 7: Print the result

Step 8: Stop the process

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

np.random.seed(0)

X = np.random.rand(100) * 10

Y = 2.5 * X + np.random.normal(0, 2, 100)

plt.scatter(X, Y, color='blue', alpha=0.7)

plt.xlabel('X')

plt.ylabel('Y')

plt.title('Scatter Plot: X vs Y')

plt.show()
```

```
X = X.reshape(-1, 1)

model = LinearRegression()

model.fit(X, Y)

slope = model.coef_[0]

intercept = model.intercept_

print(f"Slope (beta_1): {slope}")

print(f"Intercept (beta_0): {intercept}")

Y_pred = model.predict(X)

plt.scatter(X, Y, color='blue', alpha=0.7, label='Data')

plt.plot(X, Y_pred, color='red', label='Fitted Line')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('Simple Linear Regression: Fitted Line')

plt.legend()

plt.show()

r_squared = model.score(X, Y)

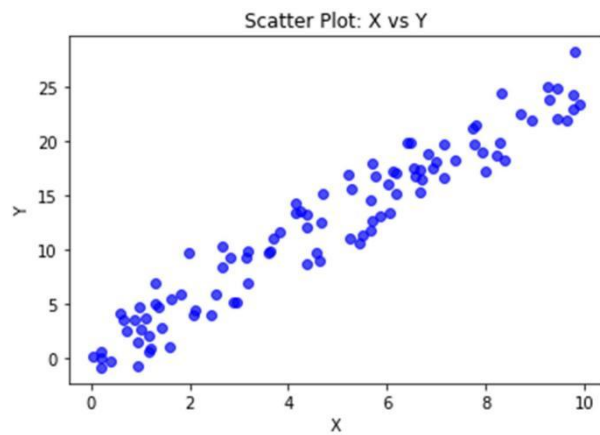
print(f"R-squared: {r_squared}")

X_new = np.array([[15]])

Y_new = model.predict(X_new)

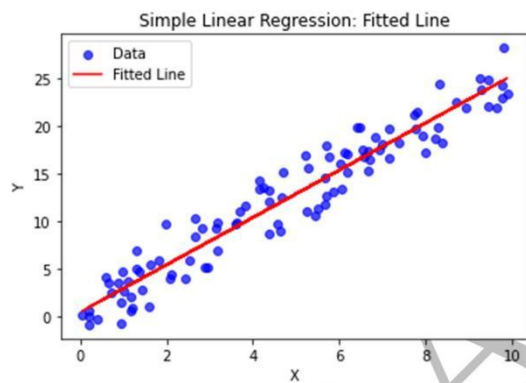
print(f"Predicted Y for X = 15: {Y_new[0]}")
```

OUTPUT:



Slope (beta_1): 2.487387004280408

Intercept (beta_0): 0.4443021548944568



R-squared: 0.928337996765404

Predicted Y for X = 15: 37.75510721910058

RESULT:

Thus the computation for Simple Linear Regression was successfully completed.

EXP NO: 6	Z-test
Date:	

AIM:

To write a python program for Z-test

ALGORITHM:

Step 1: Start the Program

Step 2: Import math package

Step 3: Define Z-test function

Step 4: Calculate Z-test using formula

Step 5: Print the result

Step 6: Stop the process

PROGRAM:

```
import numpy as np
import scipy.stats as stats

mean_1 = 50
mean_2 = 45
std_1 = 10
std_2 = 12
size_1 = 40
size_2 = 35

z_score_two_sample = (mean_1 - mean_2) / np.sqrt((std_1**2 / size_1) + (std_2**2 / size_2))
p_value_two_sample = 2 * (1 - stats.norm.cdf(abs(z_score_two_sample)))
print(f'Z-Score: {z_score_two_sample}')
print(f'P-value: {p_value_two_sample}')
```

OUTPUT:

Z-Score: 1.9441444452997994
P-value: 0.051878034893831915

RESULT:

Thus the computation for Z-test was successfully completed.

EXP NO: 7	T-test
Date:	

AIM:

To write a python program for T-test

ALGORITHM:

Step 1: Start the Program

Step 2: Import math package

Step 3: Define T-test function

Step 4: Calculate T-test using formula

Step 5: Print the result

Step 6: Stop the process

PROGRAM:

```
import scipy.stats as stats

import numpy as np

sample_data = np.array([52, 55, 48, 49, 53, 54, 51, 50, 55, 58, 56, 57, 52, 51, 54, 53, 59, 61, 50,
52, 54, 53, 49, 47, 52, 51, 50, 48, 56, 55])

population_mean = 50

t_stat, p_value = stats.ttest_1samp(sample_data, population_mean)

print(f"T-statistic: {t_stat}")

print(f"P-value: {p_value}")
```

OUTPUT:

T-statistic: 4.571679054413011
P-value: 8.327654458471987e-05

RESULT:

Thus the computation for T-test was successfully completed.

EXP NO: 8	ANOVA
Date:	

AIM:

To write a python program for ANOVA

ALGORITHM:

Step 1: Start the Program

Step 2: Import package

Step 3: Prepare the Data

Step 4: Perform ANOVA

Step 5: Calculate the F-statistic

Step 6: Calculate the P-value

Step 7: Print the result

Step 8: Stop the process

PROGRAM:

```
import numpy as np

import scipy.stats as stats

group_1 = np.array([23, 45, 67, 32, 45, 34, 43, 45, 56, 42])
group_2 = np.array([45, 32, 23, 43, 46, 32, 21, 22, 43, 43])
group_3 = np.array([65, 78, 56, 67, 82, 73, 74, 65, 68, 74])

f_stat, p_value = stats.f_oneway(group_1, group_2, group_3)

print(f"F-statistic: {f_stat}")

print(f"P-value: {p_value}")

if p_value < 0.05:

    print("There is a significant difference between the group means.")
```

else:

```
print("There is no significant difference between the group means.")
```

OUTPUT:

F-statistic: 32.6259618124822

P-value: 6.255218731829188e-08

There is a significant difference between the group means.

RESULT:

Thus the computation for ANOVA was successfully completed.

EXP NO: 9	Building and validating linear models
Date:	

AIM:

To write a python program to building and validating linear models using jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import package

Step 3: Prepare the Data

Step 4: Build the Model

Step 5: Evaluate the Model

Step 6: Model Diagnostics

Step 7: Print the result

Step 8: Stop the process

PROGRAM:

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
np.random.seed(0)
X = np.random.rand(100, 1) * 10
y = 2.5 * X.squeeze() + np.random.randn(100) * 2
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train_sm = sm.add_constant(X_train)
```

```

X_test_sm = sm.add_constant(X_test)

model = sm.OLS(y_train, X_train_sm).fit()

y_pred = model.predict(X_test_sm)

print(model.summary())

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')

print(f'R-squared: {r2}')

```

OUTPUT:

OLS Regression Results

Dep. Variable:	y	R-squared:	0.932
Model:	OLS	Adj. R-squared:	0.931
Method:	Least Squares	F-statistic:	1074.
Date:	Thu, 19 Dec 2024	Prob (F-statistic):	2.29e-47
Time:	14:52:46	Log-Likelihood:	-169.42
No. Observations:	80	AIC:	342.8
Df Residuals:	78	BIC:	347.6
Df Model:	1		
Covariance Type:	nonrobust		

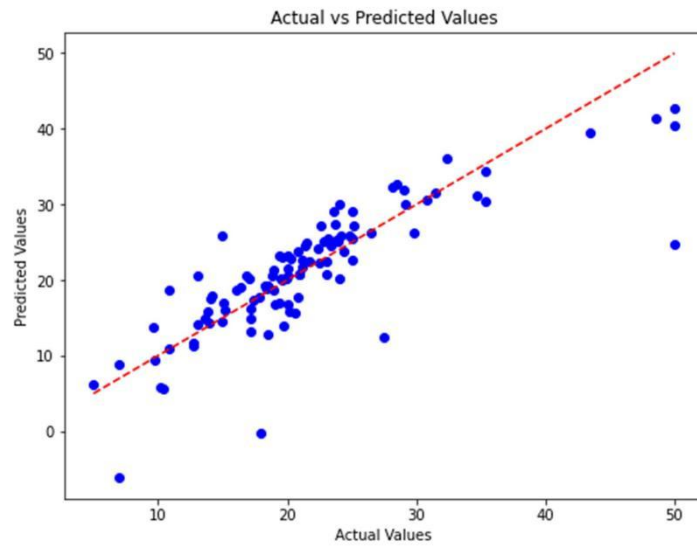
	coef	std err	t	P> t	[0.025	0.975]
const	0.4127	0.417	0.990	0.325	-0.417	1.242
x1	2.4961	0.076	32.776	0.000	2.344	2.648

Omnibus:	8.580	Durbin-Watson:	2.053
Prob(Omnibus):	0.014	Jarque-Bera (JB):	3.170
Skew:	0.107	Prob(JB):	0.205
Kurtosis:	2.048	Cond. No.	10.3

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Mean Squared Error: 3.6710129878857174

R-squared: 0.896480483165161



RESULT:

Thus the computation for building and validating linear models was successfully completed.

EXP NO: 10	Building and validating logistic models
Date:	

AIM:

To write a python program to building and validating logistic models using jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import python libraries

Step 3: Generate synthetic data

Step 4: Split the data

Step 5: Build the logistic regression model

Step 6: Make predictions and Evaluate the model

Step 7: Print evaluation metrics and Print the result

Step 8: Stop the process

PROGRAM:

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
np.random.seed(0)
```

```
X = np.random.rand(100, 2)
```

```
y = (X[:, 0] + X[:, 1] > 1).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

conf_matrix = confusion_matrix(y_test, y_pred)

class_report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')

print('Confusion Matrix:')

print(conf_matrix)

print('Classification Report:')

print(class_report)

plt.figure(figsize=(10, 6))

plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm', edgecolors='k', s=100,
            label='True Labels')

plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, marker='x', cmap='coolwarm', s=100,
            label='Predicted Labels')

plt.title('Logistic Regression Predictions')

plt.xlabel('Feature 1')

plt.ylabel('Feature 2')

plt.legend()

plt.show()
```

OUTPUT:

Accuracy: 0.9

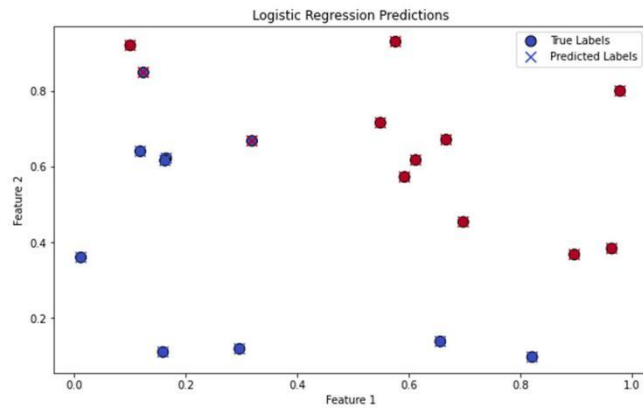
Confusion Matrix:

```
[[ 8  2]
 [ 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	0.80	0.89	10
1	0.83	1.00	0.91	10
accuracy			0.90	20
macro avg	0.92	0.90	0.90	20
weighted avg	0.92	0.90	0.90	20



RESULT:

Thus the computation for building and validating logistic models was successfully completed.

EXP NO: 11	Time series analysis
Date:	

AIM:

To write a python program to time series analysis using jupyter notebook.

ALGORITHM:

Step 1: Start the Program

Step 2: Import python libraries

Step 3: Generate a time series data

Step 4: Create a DataFrame

Step 5: Print the result

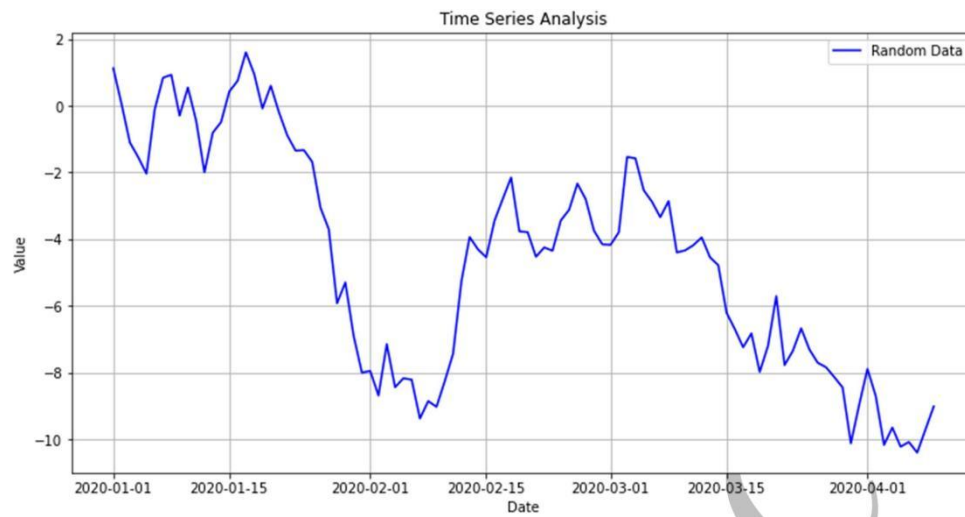
Step 6: Stop the process

PROGRAM:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
date_range = pd.date_range(start='1/1/2020', periods=100)
data = np.random.randn(100).cumsum()
time_series_data = pd.DataFrame(data, index=date_range, columns=['Value'])
plt.figure(figsize=(12, 6))
plt.plot(time_series_data.index, time_series_data['Value'], label='Random Data', color='blue')
plt.title('Time Series Analysis')
plt.xlabel('Date')
plt.ylabel('Value')
plt.legend()
plt.grid()
```

```
plt.show()
```

OUTPUT:



RESULT:

Thus the computation for time series analysis was successfully completed.